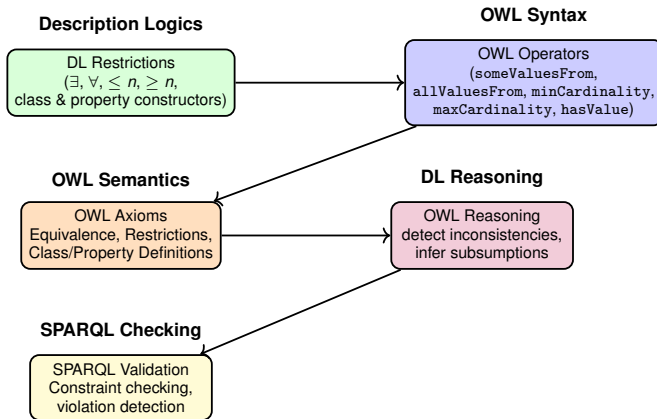


OWL & Constraints (Lecture 7)



Lecture 7



Lecture 7: From DL restrictions and OWL operators to reasoning & SPARQL-based validation of constraints

Agenda

- Restrictions in Description Logics (DLs)
- OWL
 - Operators to represent restrictions
 - Axioms
- SPARQL queries checking restrictions

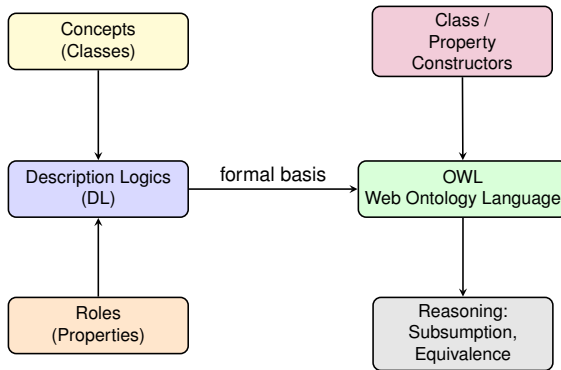
Description Logics (From Lecture 6)

Description Logics

Description Logics (DL):

- A restricted first-order language.
- Allows for the representation of knowledge in terms of:
 - Terminological (abstract) concepts
 - Instances or individuals of abstract concepts
 - Subsumption relationships
 - Equivalence
 - Class construction
 - Property construction

Intuition: OWL and Description Logics



DL provides the logical foundation; OWL adds constructors, vocabulary, and machine reasoning support.

Description Logics (DL) (Lecture 6)

A Theory in DL

A theory (knowledge base) in DL is composed of:

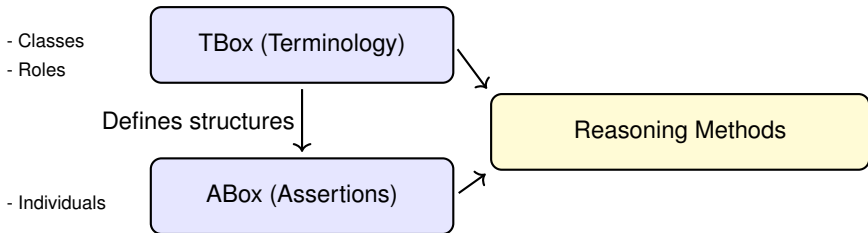
- TBox: Terminology or vocabulary
 - Classes/Concepts: Denote sets of individuals.
 - Roles: Binary relations between individuals.
- ABox: Set of statements about individuals.

Description Logics (DL) (Lecture 6)

A Theory in DL

A theory (knowledge base) in DL is composed of:

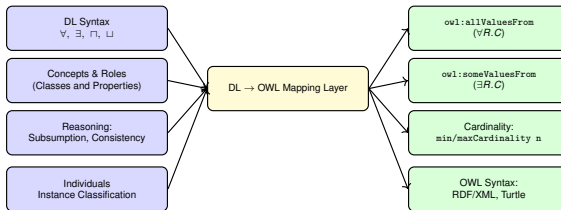
- TBox: Terminology or vocabulary
 - Classes/Concepts: Denote sets of individuals.
 - Roles: Binary relations between individuals.
- ABox: Set of statements about individuals.



DL Integrity Restrictions

Operator	Description	Example
$\forall R.T$	Instances in R related to only entities in T	$\forall \text{ex:takes_part.Researcher_MD}$
$\exists R.T$	Instances in R related to at least one entity in T	$\exists \text{ex:takes_part.Senior_Researcher_MD}$
$\exists R.\{x_1\}$	Existential quantifier in an enumerated set	$\exists \quad \text{ex:coordinates}\{ \text{ex:JohnSmith}, \text{ex:MarySmith} \}$
$\geq nR.T$	Instances in R related to at least n entities in T	$\geq 2 \text{ex:coordinates.Research_project}$
$\leq nR.T$	Instances in R related to at most n entities in T	$\leq 2 \text{ex:coordinates.Research_project}$
$= nR.T$	Instances in R related to exactly n entities in T	$= 2 \text{ex:coordinates.Research_project}$

Relationship between Description Logics and OWL



DL provides the theory and operators; OWL provides the Web-ready implementation and vocabulary.

Relationship between Description Logics and OWL

Key Concepts in Description Logics (DL):

- DL provides a formal foundation for knowledge representation.
- Syntax is based on logical operators ($\forall$, $\exists$, $\sqcap$, $\sqcup$) and quantifiers.
- Focuses on defining and reasoning over classes (concepts), properties (roles), and individuals.
- Enables inference tasks like subsumption, consistency checking, and instance classification.

Mapping to OWL:

- OWL (Web Ontology Language) is built on DL for practical use in semantic web applications.
- DL axioms are expressed using OWL constructs:
 - $\forall R.C \rightarrow \text{owl:allValuesFrom } C.$
 - $\exists R.C \rightarrow \text{owl:someValuesFrom } C.$
 - $\leq nR \rightarrow \text{owl:maxCardinality } n.$
 - $\geq nR \rightarrow \text{owl:minCardinality } n.$
- OWL adds RDF/XML syntax for web implementation.

Relationship between Description Logics and OWL

Key Concepts in Description Logics (DL):

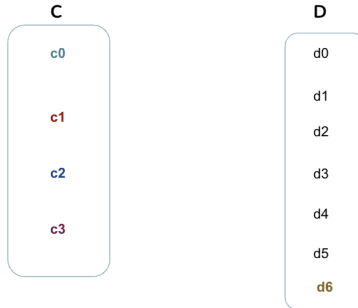
- DL provides a formal foundation for knowledge representation.
- Syntax is based on logical operators ($\forall$, $\exists$, $\sqcap$, $\sqcup$) and quantifiers.
- Focuses on defining and reasoning over classes (concepts), properties (roles), and individuals.
- Enables inference tasks like subsumption, consistency checking, and instance classification.

Mapping to OWL:

- OWL (Web Ontology Language) is built on DL for practical use in semantic web applications.
- DL axioms are expressed using OWL constructs:
 - $\forall R.C \rightarrow \text{owl:allValuesFrom } C.$
 - $\exists R.C \rightarrow \text{owl:someValuesFrom } C.$
 - $\leq nR \rightarrow \text{owl:maxCardinality } n.$
 - $\geq nR \rightarrow \text{owl:minCardinality } n.$
- OWL adds RDF/XML syntax for web implementation.

- Description Logics are the theoretical basis for OWL.
- OWL extends DL with web-compatible features (e.g., RDF syntax) and practical tools.

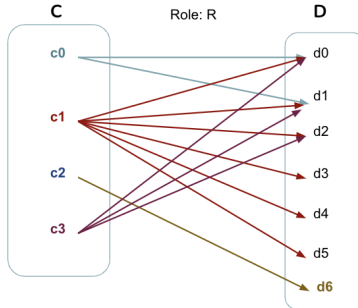
Classes in Description Logics



Classes represent group of individuals:

- **Class C:** Contains individuals c_0 , c_1 , c_2 , c_3 .
- **Class D:** Contains individuals d_0 , d_1 , d_2 , d_3 , d_4 , d_5 , d_6 .

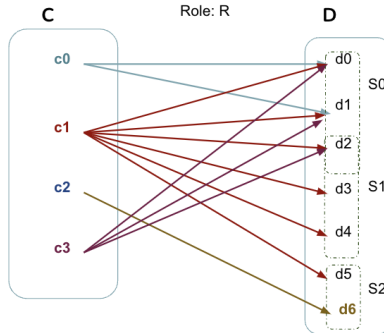
Roles in Description Logics (1/3)



Roles represent binary relationships between individuals of two classes.

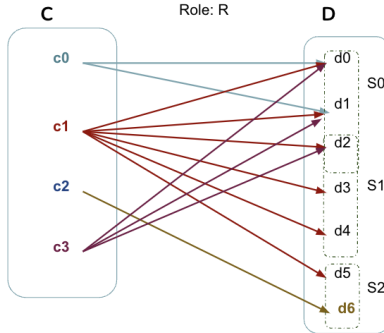
- **Class C:** Contains individuals c_0, c_1, c_2, c_3 .
- **Class D:** Contains individuals $d_0, d_1, d_2, d_3, d_4, d_5, d_6$.
- **Role R :** Represents relationships between individuals in C and D , as shown by the arrows.

Roles in Description Logics (2/3)



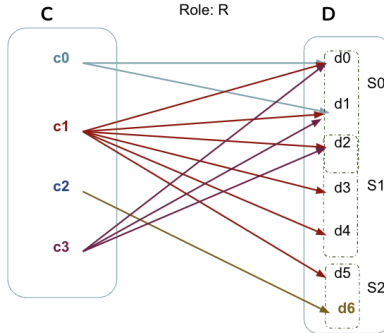
- All the elements in $R(c0)$ are of type S0
- Some of the elements in $R(c1)$ are of type: S0, S1, S2
- All the elements in $R(c2)$ are of type S2
- All the elements in $R(c3)$ are of type S0

Roles in Description Logics (3/3)



- $\{c0, c3\}$ is the set, such as, **for every** $c_i \in C$, $R(c_i)$ is of type $S0$.
- $\{c0, c1, c3\}$ is the set, such as, **at least one** $c_i \in C$, $R(c_i)$ is of type $S0$.
- $\{c2\}$ is the set, such as, **all** $c_i \in C$, $R(c_i)$ is of type $S2$.

Restrictions in DL



- $\{c0, c3\}$ is the set, such as, **for every** $c_i \in C$, $R(c_i)$ is of type $S0$. This is represented by $\forall R.S0$
- $\{c0, c1, c3\}$ is the set, such as, **at least one** $c_i \in C$, $R(c_i)$ is of type $S0$. This is represented by $\exists R.S0$
- $\{c2\}$ is the set, such as, **all** $c_i \in C$, $R(c_i)$ is of type $S2$. This is represented by $\forall R.S2$

Another Example of Restrictions in DL (1/2)

#Classes

```
ex:ParentLivingBeing a rdfs:Class.
ex:LivingBeing a rdfs:Class.
ex:Ogre a rdfs:Class.
ex:FictionalCharacter a rdfs:Class.
ex:Human a rdfs:Class.
```

#SubClasses

```
ex:Ogre rdfs:subClassOf ex:LivingBeing.
ex:FictionalCharacter rdfs:subClassOf
    ex:LivingBeing.
ex:Human rdfs:subClassOf
    ex:LivingBeing.
ex:ParentLivingBeing rdfs:subClassOf
    ex:LivingBeing.
```

#Properties

```
ex:hasChild a rdf:Property;
    rdfs:domain ex:ParentLivingBeing;
    rdfs:range ex:LivingBeing.
```

#Instances of Classes

```
ex:c0 a ex:ParentLivingBeing.
ex:c1 a ex:ParentLivingBeing.
ex:c2 a ex:ParentLivingBeing.
ex:c3 a ex:ParentLivingBeing.
```

```
ex:d0 a ex:Ogre.
ex:d1 a ex:Ogre.
ex:d2 a ex:Ogre.
ex:d2 a ex:FictionalCharacter.
ex:d3 a ex:FictionalCharacter.
ex:d4 a ex:FictionalCharacter.
ex:d5 a ex:Human.
ex:d6 a ex:Human.
```

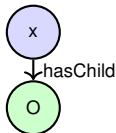
#Instances of Properties

```
ex:c0 ex:hasChild ex:d0, ex:d1.
ex:c1 ex:hasChild ex:d0, ex:d1, ex:d2,
    ex:d3, ex:d4, ex:d5.
ex:c2 ex:hasChild ex:d6.
ex:c3 ex:hasChild ex:d0, ex:d1, ex:d2.
```

Intuition: DL Restrictions on ex:hasChild

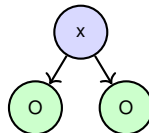
$\exists \text{ hasChild.Ogre}$

$\forall \text{ hasChild.Ogre}$



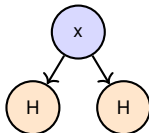
Satisfies: $\{\text{ex:c0}, \text{ex:c1}, \text{ex:c3}\}$

$\forall \text{ hasChild.Human}$

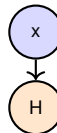


Satisfies: $\{\text{ex:c0}, \text{ex:c3}\}$

$\exists \text{ hasChild.Human}$



Satisfies: $\{\text{ex:c2}\}$



Satisfies: $\{\text{ex:c1}\}$

$\exists$: requires at least one matching child | $\forall$: requires all children belong to the class

Another Example of Restrictions in DL (2/2)

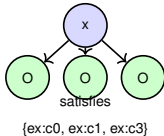
Restriction	Logical Formula	Set of Individuals
Individuals who have at least one Ogre child	$\exists \text{ex:hasChild.ex:Ogre}$	$\{\text{ex:c0}, \text{ex:c1}, \text{ex:c3}\}$
Individuals whose children are all Ogres	$\forall \text{ex:hasChild.ex:Ogre}$	$\{\text{ex:c0}, \text{ex:c3}\}$
Individuals whose children are all Humans	$\forall \text{ex:hasChild.ex:Human}$	$\{\text{ex:c2}\}$
Individuals who have at least one Human child	$\exists \text{ex:hasChild.ex:Human}$	$\{\text{ex:c1}\}$

Key Concepts:

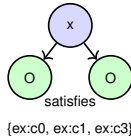
- $\exists$: Existential quantifier, indicating the existence of at least one individual.
- $\forall$: Universal quantifier, indicating all individuals satisfy the condition.

Intuition: Cardinality Restrictions in DL

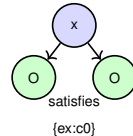
≤ 3 hasChild.Ogre



≥ 2 hasChild.Ogre



$= 2$ hasChild.Ogre



Cardinality restrictions constrain *how many* role fillers an individual may have.

Cardinality Restrictions in DL

Restriction	Formula	Instances
Individuals who have at most three Ogre children	$\leq 3 \text{hasChild.Ogre}$	$\{\text{ex:c0}, \text{ex:c1}, \text{ex:c3}\}$
Individuals who have at least two Ogre children	$\geq 2 \text{hasChild.Ogre}$	$\{\text{ex:c0}, \text{ex:c1}, \text{ex:c3}\}$
Individuals who have exactly two Ogre children	$= 2 \text{hasChild.Ogre}$	$\{\text{ex:c0}\}$

Restrictions in OWL

OWL Predicates to Represent Restrictions

- **owl:allValuesFrom:** Defines a class that contains all the resources that have values for a given property that are all of a certain type.
- **owl:someValuesFrom:** Defines a class that contains the resources that have at least one value for a given property of a certain type.
- **owl:hasValue:** Defines a class that contains all the resources that have a specific value for a given property.
- **Qualified cardinality restriction:** Defines classes with a given number of values for a property in a given class.

DL Restrictions in OWL

DL Operator	OWL Syntax
$\forall R.T$	C_1 owl:allValuesFrom T ; owl:onProperty R
$\exists R.T$	C_1 owl:someValuesFrom T ; owl:onProperty R
$\geq n R.T$	C_1 owl:minCardinality n ; owl:onProperty R
$\leq n R.T$	C_1 owl:maxCardinality n ; owl:onProperty R
$= n R.T$	C_1 owl:cardinality n ; owl:onProperty R
$\exists R.\{a\}$	C_1 owl:hasValue a ; owl:onProperty R

Notation in OWL and Turtle (1/2)

OWL Syntax for All Values From ($\forall$)

Class `ex:C1` is equivalent to the class composed of the resources that have values for property `R` that are all in the class `T`.

- **OWL Syntax:**

```
ex:C1 owl:equivalentClass [owl:allValuesFrom T ;  
  owl:onProperty R].
```

- **Turtle Serialization:**

```
ex:C1 owl:equivalentClass _:b .  
_:b owl:allValuesFrom T .  
_:b owl:onProperty R .
```

Notation in OWL and Turtle (2/2)

OWL Syntax for Some Values From ($\exists$)

Class `ex:C1` is equivalent to the class composed of the resources that have at least one value for the property `R` of the class `T`.

- **OWL Syntax:**

```
ex:C1 owl:equivalentClass [owl:someValuesFrom T ;  
    owl:onProperty R].
```

- **Turtle Serialization:**

```
ex:C1 owl:equivalentClass _:b .  
_:b owl:someValuesFrom T .  
_:b owl:onProperty R .
```

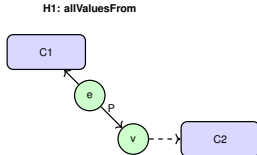
Restrictions in DL and OWL

Restriction	DL Formula	OWL Expression
Individuals who have at least one Ogre child	$\exists \text{ex:hasChild.ex:Ogre}$	<code>ex:C1 owl:equivalentClass [owl:someValuesFrom ex:Ogre ; owl:onProperty ex:hasChild] .</code>
Individuals whose children are all Ogres	$\forall \text{ex:hasChild.ex:Ogre}$	<code>ex:C1 owl:equivalentClass [owl:allValuesFrom ex:Ogre ; owl:onProperty ex:hasChild] .</code>
Individuals whose children are all Humans	$\forall \text{ex:hasChild.ex:Human}$	<code>ex:C1 owl:equivalentClass [owl:allValuesFrom ex:Human ; owl:onProperty ex:hasChild] .</code>
Individuals who have at least one Human child	$\exists \text{ex:hasChild.ex:Human}$	<code>ex:C1 owl:equivalentClass [owl:someValuesFrom ex:Human ; owl:onProperty ex:hasChild] .</code>

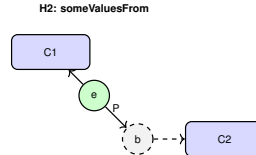
Key Concepts:

- $\exists$: Existential quantifier, indicating the existence of at least one individual.
- $\forall$: Universal quantifier, indicating all individuals satisfy the condition.
- OWL expressions use `owl:someValuesFrom` and `owl:allValuesFrom` to represent existential and universal quantifiers.

Intuition: Quantified Value Restrictions in OWL



If e is a $C1$ and all its P -values must be $C2$
then v is inferred to be a $C2$.



If e is a $C1$ and must have some P -value in $C2$
then a new anonymous individual b is created and its type is $C2$.

Universal ($\forall$) pushes constraints to existing values. Existential ($\exists$) creates a value ensuring a restriction is satisfied.

More Axioms

Group H (Quantified Values)

H1. Universal Quantification:

IF (C1, owl:allValuesFrom, C2)

AND (C1, owl:onProperty, P)

AND (e, rdf:type, C1) **AND** (e, P, v)

THEN (v, rdf:type, C2)

H2. Existential Quantification:

IF (C1, owl:someValuesFrom, C2)

AND (C1, owl:onProperty, P)

AND (e, rdf:type, C1)

THEN (e, P, _ : b) **AND** (_ : b, rdf:type, C2)

Example to Illustrate Axioms

■ OWL Definition

```
ex:OgreParent owl:equivalentClass
    [owl:allValuesFrom ex:Ogre;
     owl:onProperty ex:hasChild].
ex:OgreMom rdfs:subClassOf ex:OgreParent.
ex:OgreParent rdfs:subClassOf ex:Ogre.
```

■ Individuals

```
ex:ogreFiona a ex:OgreMom;
    ex:hasChild ex:Felicia, ex:Fergus, ex:Farkle.
```



ex:ogreFiona

Example to Illustrate Axioms

■ OWL Definition

```
ex:OgreParent owl:equivalentClass
    [owl:allValuesFrom ex:Ogre;
     owl:onProperty ex:hasChild].
ex:OgreMom rdfs:subClassOf ex:OgreParent.
ex:OgreParent rdfs:subClassOf ex:Ogre.
```

■ Individuals

```
ex:ogreFiona a ex:OgreMom;
    ex:hasChild ex:Felicia, ex:Fergus, ex:Farkle.
```

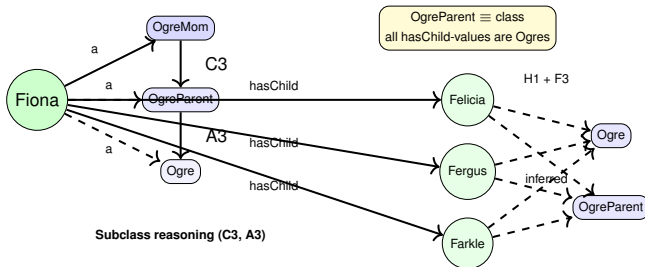
Inferred Triples:

```
ex:Felicia    a ex:Ogre.
ex:Fergus     a ex:Ogre.
ex:Farkle     a ex:Ogre.
ex:OgreMom    rdfs:subClassOf ex:Ogre.
ex:ogreFiona  a ex:OgreParent, ex:Ogre.
```



ex:ogreFiona

Visualization of Inferred Facts from Learned Axioms



Subclass axioms propagate Fiona's type; universal restriction propagates Ogre/OgreParent to her children.

Inferred Facts from Axioms

Relevant Axioms

■ C3: SubClassOf

- IF (C1 rdfs:subClassOf C2) AND
(ex:a ex:P1 ex:b)
THEN (ex:a ex:P2 ex:b).

■ F3: Equivalence Class Instance

- IF (C1 owl:equivalentClass C2)
AND (x rdf:type C1)
THEN (x rdf:type C2)

■ H1: Universal Quantifier

- IF (C1 owl:allValuesFrom C2) AND
(C1 owl:onProperty P) AND
(e rdf:type C1) AND (e P v)
THEN (e, rdf:type, C2)

What are title, year, and research field label of the papers that describe at least one dataset?

■ A3: SubClass Instance

- IF (C1 rdfs:subClassOf C2) AND
(e rdf:type C1)
THEN (e rdf:type C2)

Inferred Facts from Axioms

Relevant Axioms

■ C3: SubClassOf

- IF (C1 rdfs:subClassOf C2) AND
(ex:a ex:P1 ex:b)
THEN (ex:a ex:P2 ex:b).

■ F3: Equivalence Class Instance

- IF (C1 owl:equivalentClass C2)
AND (x rdf:type C1)
THEN (x rdf:type C2)

■ H1: Universal Quantifier

- IF (C1 owl:allValuesFrom C2) AND
(C1 owl:onProperty P) AND
(e rdf:type C1) AND (e P v)
THEN (e, rdf:type, C2)

What are title, year, and research field label of the papers that describe at least one dataset?

■ A3: SubClass Instance

- IF (C1 rdfs:subClassOf C2) AND
(e rdf:type C1)
THEN (e rdf:type C2)

■ Step 1: From the triples

```
ex:ogreFiona a ex:OgreMom.  
ex:OgreMom rdfs:subClassOf ex:OgreParent  
infer using C3:
```

```
ex:ogreFiona a ex:OgreParent.
```

■ Step 2: Using F3, H1, and A3

```
ex:OgreParent owl:equivalentClass _:b.  
_:b owl:allValuesFrom ex:Ogre.  
_:b owl:onProperty ex:hasChild.  
ex:ogreFiona a ex:OgreParent.  
ex:ogreFiona ex:hasChild ex:Felicia,  
ex:Fergus,  
ex:Farkle.  
ex:OgreParent rdfs:subClassOf ex:Ogre.
```

infer:

```
ex:Felicia a ex:OgreParent, ex:Ogre.  
ex:Fergus a ex:OgreParent, ex:Ogre.  
ex:Farkle a ex:OgreParent, ex:Ogre.
```

Intuition: Existential Restrictions (Group H)

H3: Inferring Class Membership

TBox:

C_1 has the restriction:
owl:someValuesFrom C_2
owl:onProperty P

ABox:

(e, P, v)
 $(v, \text{rdf:type}, C_2)$

H3 Inference:

If e is related via P to an instance of C_2 ,
then e must be an instance of C_1 :

$(e, \text{rdf:type}, C_1)$

"If e has some P -filler of type C_2 , then e is in C_1 ."

H4: Comparing Existential Restrictions

TBox:

C_1 restricted by $\exists P.C_2$
 C_3 restricted by $\exists P.C_4$
 $(C_2, \text{rdfs:subClassOf}, C_4)$

Intuition:

"Having some P from C_2 " is more specific
than "having some P from C_4 ".

H4 Inference:

$(C_1, \text{rdfs:subClassOf}, C_3)$

"More specific restriction, then more specific
class."

More Axioms

Group H (Quantified Values)

H3. Instantiation Existential Quantification:

IF (C1, owl:someValuesFrom, C2)

AND (C1, owl:onProperty, P)

AND (e, P, v) **AND** (v, rdf:type, C2)

THEN (e, rdf:type, C1)

H4. Existential Quantification and SubClass:

IF (C1, owl:someValuesFrom, C2) **AND** (C1, owl:onProperty, P)

AND (C3, owl:someValuesFrom, C4) **AND** (C3, owl:onProperty, P)

AND (C2, rdfs:subClassOf, C4)

THEN (C1, rdfs:subClassOf, C3)

Examples of Axioms: Group H

H3. Instantiation Existential Quantification:

- **Axiom:** If (C1, owl:someValuesFrom, C2) AND (C1, owl:onProperty, P) AND (e, P, v) AND (v, rdf:type, C2), THEN (e, rdf:type, C1).
- **Example:**
ex:Parent owl:someValuesFrom ex:Child ;
 owl:onProperty ex:hasChild.
ex:Felicia rdf:type ex:Child.
ex:ogreFiona ex:hasChild ex:Felicia.
- **Inferred:** ex:ogreFiona rdf:type ex:Parent.

Examples of Axioms: Group H

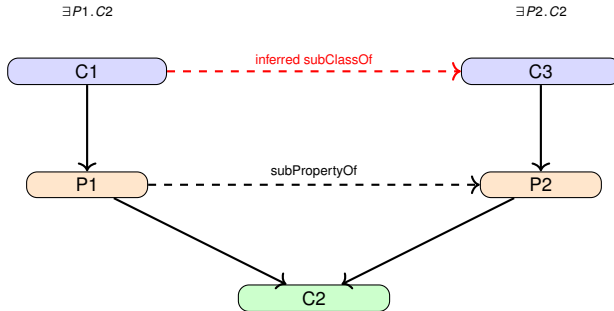
H3. Instantiation Existential Quantification:

- **Axiom:** If (C1, owl:someValuesFrom, C2) AND (C1, owl:onProperty, P) AND (e, P, v) AND (v, rdf:type, C2), THEN (e, rdf:type, C1).
- **Example:**
ex:Parent owl:someValuesFrom ex:Child ;
 owl:onProperty ex:hasChild.
ex:Felicia rdf:type ex:Child.
ex:ogreFiona ex:hasChild ex:Felicia.
- **Inferred:** ex:ogreFiona rdf:type ex:Parent.

H4. Existential Quantification and SubClass:

- **Axiom:** If (C1, owl:someValuesFrom, C2) AND (C1, owl:onProperty, P) AND (C3, owl:someValuesFrom, C4) AND (C3, owl:onProperty, P) AND (C2, rdfs:subClassOf, C4), THEN (C1, rdfs:subClassOf, C3).
- **Example:**
ex:Parent owl:someValuesFrom ex:Ogre ;
 owl:onProperty ex:hasChild.
ex:GrandParent owl:someValuesFrom ex:LivingBeing ;
 owl:onProperty ex:hasChild.
ex:Ogre rdfs:subClassOf ex:LivingBeing.
- **Inferred:** ex:Parent rdfs:subClassOf ex:GrandParent.

Intuition: Axiom H5



If both C1 and C3 require some value in C2, and P1 is a subproperty of P2, then C1 must be a subclass of C3.

More Axioms of Group H

Group H (Quantified Values)

H5. Existential Quantification and SubProperty:

IF (C1, owl:someValuesFrom, C2)

AND (C1, owl:onProperty, P1)

AND (C3, owl:someValuesFrom, C2)

AND (C3, owl:onProperty, P2)

AND (P1, rdfs:subPropertyOf, P2)

THEN (C1, rdfs:subClassOf, C3)

Example of H5

H5. Existential Quantification and SubProperty:

- **Axiom:** If (C1, owl:someValuesFrom, C2) AND (C1, owl:onProperty, P1) AND (C3, owl:someValuesFrom, C2) AND (C3, owl:onProperty, P2) AND (P1, rdfs:subPropertyOf, P2), THEN (C1, rdfs:subClassOf, C3).

Example:

```
ex:OgreGuardian owl:someValuesFrom ex:OgreChild ;  
                owl:onProperty ex:protects.
```

```
ex:OgreParent owl:someValuesFrom ex:OgreChild ;  
              owl:onProperty ex:looksAfter.
```

```
ex:protects rdfs:subPropertyOf ex:looksAfter.
```

Inferred:

```
ex:OgreGuardian rdfs:subClassOf ex:OgreParent.
```

Explanation: Since `ex:protects` is a sub-property of `ex:looksAfter`, and `ex:OgreGuardian` has `ex:protects` for `ex:OgreChild`, it can be inferred that `ex:OgreGuardian` is a subclass of `ex:OgreParent`, which has `ex:looksAfter` for `ex:OgreChild`.

Example to Illustrate Axioms

```
ex:ogreFiona a ex:Ogre.  
ex:LiliansMother ex:hasGrandChild ex:ogreFiona ,  
                                   ex:ArthurPendragon.  
ex:HybridOgreGrandMom owl:equivalentClass  
    [owl:someValuesFrom ex:Ogre;  
     owl:onProperty ex:hasGrandChild].  
ex:hasGrandChild a rdf:Property ;  
    rdfs:domain ex:LivingBeing;  
    rdfs:range  ex:LivingBeing .  
ex:HybridOgreGrandMom rdfs:subClassOf ex:HybridOgreGrandParent.  
ex:HybridOgreGrandParent rdfs:subClassOf ex:HybridOgre.
```

Example to Illustrate Axioms

```
ex:ogreFiona a ex:Ogre.  
ex:LiliansMother ex:hasGrandChild ex:ogreFiona ,  
                                   ex:ArthurPendragon.  
ex:HybridOgreGrandMom owl:equivalentClass  
    [owl:someValuesFrom ex:Ogre;  
     owl:onProperty ex:hasGrandChild].  
ex:hasGrandChild a rdf:Property ;  
    rdfs:domain ex:LivingBeing;  
    rdfs:range  ex:LivingBeing .  
ex:HybridOgreGrandMom rdfs:subClassOf ex:HybridOgreGrandParent.  
ex:HybridOgreGrandParent rdfs:subClassOf ex:HybridOgre.
```

Inferred Facts

```
ex:LiliansMother a ex:HybridOgreGrandMom,  
                  ex:HybridOgreGrandParent,  
                  ex:HybridOgre,  
                  ex:LivingBeing.  
ex:HybridOgreGrandMom rdfs:subClassOf ex:HybridOgre.
```

Inferred Facts and Axioms in Action (1/2)

H3: SomeValuesFrom Restriction

IF (C1, owl:someValuesFrom, C2) AND
(C1, owl:onProperty, P) AND
(e, P, v) AND (v, rdf:type, C2)
THEN (e, rdf:type, C1)

A2: Transitivity of SubClassOf

IF (C1, rdfs:subClassOf, C2) AND
(C2, rdfs:subClassOf, C3)
THEN (C1, rdfs:subClassOf, C3)

A3: SubClass Instance

IF (C1, rdfs:subClassOf, C2) AND
(e, rdf:type, C1)
THEN (e, rdf:type, C2)

From the triples:

```
ex:ogreFiona a ex:Ogre.
ex:HybridOgreGrandMom owl:equivalentClass
    [owl:someValuesFrom ex:Ogre;
     owl:onProperty ex:hasGrandChild].
ex:LiliansMother ex:hasGrandChild
    ex:ogreFiona, ex:ArthurPendragon.
```

Using H3, infer:

```
ex:LiliansMother rdf:type ex:HybridOgreGrandMom.
```

From the triples:

```
ex:LiliansMother a ex:HybridOgreGrandMom.
ex:HybridOgreGrandMom rdfs:subClassOf
    ex:HybridOgreGrandParent.
ex:HybridOgreGrandParent rdfs:subClassOf ex:HybridOgre.
```

Using A2 and A3, infer:

```
ex:LiliansMother a ex:HybridOgreGrandParent,
    ex:HybridOgre.
ex:HybridOgreGrandMom rdfs:subClassOf ex:HybridOgre.
```

Inferred Facts and Axioms in Action (2/2)

D1: Property Domain

IF (P, rdfs:domain, C1) AND (a, P, b)
THEN (a, rdf:type, C1)

Relevant triples

```
ex:hasGrandChild a rdf:Property ;  
                  rdfs:domain ex:LivingBeing.  
ex:LiliansMother ex:hasGrandChild  
                  ex:ogreFiona, ex:ArthurPendragon.
```

Using D1, infer:

```
ex:LiliansMother rdf:type ex:LivingBeing.
```

Inferred Facts and Axioms in Action (2/2)

D1: Property Domain

IF (P, rdfs:domain, C1) AND (a, P, b)
THEN (a, rdf:type, C1)

Relevant triples

```
ex:hasGrandChild a rdf:Property ;  
                  rdfs:domain ex:LivingBeing.  
ex:LiliansMother ex:hasGrandChild  
                  ex:ogreFiona, ex:ArthurPendragon.
```

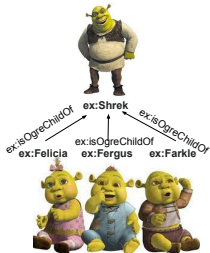
Using D1, infer:

```
ex:LiliansMother rdf:type ex:LivingBeing.
```

All Inferred Facts

```
ex:LiliansMother a ex:HybridOgreGrandMom,  
                  ex:HybridOgreGrandParent,  
                  ex:HybridOgre,  
                  ex:LivingBeing.  
ex:HybridOgreGrandMom rdfs:subClassOf ex:HybridOgre.
```

Example owl:hasValue (1/2)



```

ex:ShreksChildren owl:equivalentClass
    [owl:hasValue ex:Shrek;
     owl:onProperty ex:isOgreChildOf].
ex:Felicia ex:isOgreChildOf ex:Shrek .
ex:Fergus ex:isOgreChildOf ex:Shrek .
ex:Farkle ex:isOgreChildOf ex:Shrek .
  
```

Example owl:hasValue (1/2)

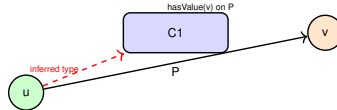


- owl:hasValue defines a class that contains all the resources that have a specific value for a given property.
- Example:
 - Property: ex:isOgreChildOf
 - Value: ex:Shrek
- The resulting class contains all individuals with this property value.

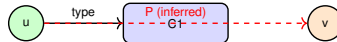
```

ex:ShreksChildren owl:equivalentClass
    [owl:hasValue ex:Shrek;
     owl:onProperty ex:isOgreChildOf].
ex:Felicia ex:isOgreChildOf ex:Shrek .
ex:Fergus ex:isOgreChildOf ex:Shrek .
ex:Farkle ex:isOgreChildOf ex:Shrek .
  
```

Intuition: owl:hasValue



I1: If $C1 \text{ owl:hasValue } v \text{ onProperty } P$ and (u, P, v) holds, then $u \text{ rdf:type } C1$



I2: If $C1 \text{ owl:hasValue } v \text{ onProperty } P$ and $u \text{ rdf:type } C1$, then the triple (u, P, v) must hold.

HasValue ties a class to a specific value: either the value gives you the class, or the class gives you the value.

More Axioms

Group I (Has Values)

I1. Filler Restriction: IF (C1, owl:hasValue, v)
AND (C1, owl:onProperty, P)
AND (u , P, v)
THEN (u , rdf:type, C1)

I2. Has Value Definition: IF (C1, owl:hasValue, v)
AND (C1, owl:onProperty, P)
AND (u , rdf:type, C1)
THEN (u , P, v)

Example owl:hasValue (2/2)

I1: Filler Restriction

IF (C1 owl:hasValue v) AND
 (C1 owl:onProperty P) AND
 (u P v)
THEN (u rdf:type C1)

From the triples:

```
_:b owl:hasValue ex:Shrek;  
      owl:onProperty ex:isOgreChildOf.  
ex:ShreksChildren owl:equivalentClass _:b.  
ex:Felicia ex:isOgreChildOf ex:Shrek .  
ex:Fergus ex:isOgreChildOf ex:Shrek .  
ex:Farkle ex:isOgreChildOf ex:Shrek .
```

Using I1, infer:

```
ex:Felicia rdf:type _:b.  
ex:Fergus rdf:type _:b.  
ex:Farkle rdf:type _:b.
```

Example owl:hasValue (2/2)

I1: Filler Restriction

```
IF (C1 owl:hasValue v) AND
  (C1 owl:onProperty P) AND
  (u P v)
THEN ( u rdf:type C1)
```

From the triples:

```
_:b owl:hasValue ex:Shrek;
    owl:onProperty ex:isOgreChildOf.
ex:ShreksChildren owl:equivalentClass _:b.
ex:Felicia ex:isOgreChildOf ex:Shrek .
ex:Fergus ex:isOgreChildOf ex:Shrek .
ex:Farkle ex:isOgreChildOf ex:Shrek .
```

Using I1, infer:

```
ex:Felicia rdf:type _:b.
ex:Fergus rdf:type _:b.
ex:Farkle rdf:type _:b.
```

F3. Equivalent Class Instances:

```
IF (C1, owl:equivalentClass,C2) AND
  ( x, rdf:type, C1) AND
THEN ( x, rdf:type, C2)
```

From the triples:

```
ex:Felicia rdf:type _:b.
ex:Fergus rdf:type _:b.
ex:Farkle rdf:type _:b.
ex:ShreksChildren owl:equivalentClass _:b.
```

Using F3, infer:

```
ex:Felicia rdf:type ex:ShreksChildren.
ex:Fergus rdf:type ex:ShreksChildren.
ex:Farkle rdf:type ex:ShreksChildren.
```

Cardinality Restrictions in OWL (1/2)

Key Points:

- **Unqualified Restrictions:**
Cardinality Restrictions apply to all values of the property.
- **Qualified Restrictions:**
Cardinality Restrictions apply to entities of a specific class that are in the object position of the constrained property.

Cardinality Restrictions in OWL (2/2)

Unqualified Cardinality Restrictions

- `owl:maxCardinality n`:
 - Restricts a property to have at **most** n values.
 - Example:
 - `C owl:maxCardinality "1"^^xsd:nonNegativeInteger;`
 - `owl:onProperty ex:hasChild.`
 - Individuals in C can have at most one `ex:hasChild`.
- `owl:minCardinality n`:
 - Requires a property to have at **least** n values.
 - Example:
 - `C owl:minCardinality "2"^^xsd:nonNegativeInteger;`
 - `owl:onProperty ex:hasSibling.`

Cardinality Restrictions in OWL (2/2)

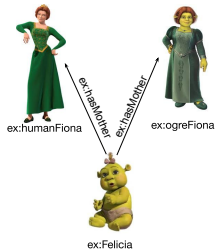
Unqualified Cardinality Restrictions

- `owl:maxCardinality n`:
 - Restricts a property to have **at most n** values.
 - Example:
 - `C owl:maxCardinality "1"^^xsd:nonNegativeInteger;`
 - `owl:onProperty ex:hasChild.`
 - Individuals in *C* can have at most one `ex:hasChild`.
- `owl:minCardinality n`:
 - Requires a property to have **at least n** values.
 - Example:
 - `C owl:minCardinality "2"^^xsd:nonNegativeInteger;`
 - `owl:onProperty ex:hasSibling.`

Qualified Cardinality Restrictions

- `owl:maxQualifiedCardinality n`:
 - Restricts a property to have **at most n** values of a specific class.
 - Example:
 - `C owl:maxQualifiedCardinality "1"^^xsd:nonNegativeInteger;`
 - `owl:onClass ex:Human;`
 - `owl:onProperty ex:hasChild.`
 - Individuals in *C* can have **at most one** child of type `ex:Human`.
- `owl:minQualifiedCardinality n`:
 - Requires a property to have **at least n** values of a specific class.
 - Example:
 - `C owl:minQualifiedCardinality "2"^^xsd:nonNegativeInteger;`
 - `owl:onClass ex:Ogre;`

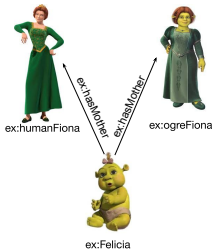
Example owl:maxCardinality (1/2)



```

ex:Child owl:equivalentClass
  [owl:maxCardinality "1"^^xsd:nonNegativeInteger;
   owl:onProperty ex:hasMother].
ex:Felicia ex:hasMother ex:humanFiona.
ex:Felicia ex:hasMother ex:ogreeFiona.
ex:Felicia a ex:Child.
  
```

Example owl:maxCardinality (1/2)

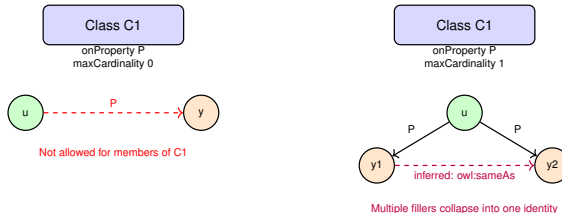


```

ex:Child owl:equivalentClass
  [owl:maxCardinality "1"^^xsd:nonNegativeInteger;
   owl:onProperty ex:hasMother].
ex:Felicia ex:hasMother ex:humanFiona.
ex:Felicia ex:hasMother ex:ogreFiona.
ex:Felicia a ex:Child.
  
```

- owl:maxCardinality imposes a restriction on the maximum number of values for a given property.
- Example:
 - Property: ex:hasMother
 - Maximum Cardinality: 1
- The resulting class contains all individuals who have **at most one value** for the given property.
- If an individual has multiple values for this property, triples with owl:sameAs must be inferred.

Intuition: Cardinality Restrictions in OWL



Max 0: property P is not allowed for members of this class. | Max 1: if two different P-values appear, OWL concludes they refer to the same individual.

More Axioms

Group J (Cardinalities)

J1. Max Cardinality Zero:

IF (C1, owl:maxCardinality, "0"^^xsd:nonNegativeInteger)
AND (C1, owl:onProperty, P)
AND (u , rdf:type, C1) **AND** (u , P, y)
THEN False

J2. Max Cardinality One:

IF (C1, owl:maxCardinality, "1"^^xsd:nonNegativeInteger)
AND (C1, owl:onProperty, P) **AND** (u , rdf:type, C1)
AND (u , P, $y1$) **AND** (u , P, $y2$)
THEN ($y1$, owl:sameAs, $y2$)

Example owl:maxCardinality (2/2)

F3. Equivalent Class Instances:

```
IF (C1, owl:equivalentClass,C2) AND
  ( x, rdf:type, C1) AND
THEN ( x, rdf:type, C2)
```

J2: Max Cardinality One:

```
IF (C1, owl:maxCardinality,
    "1"^^xsd:nonNegativeInteger)
  AND
  (C1, owl:onProperty, P) AND
  (u, rdf:type, C1) AND
  (u, P, y1) AND (u, P, y2)
THEN (y1, owl:sameAs, y2)
```

From the triples:

```
_:c owl:maxCardinality "1"^^xsd:nonNegativeInteger;
    owl:onProperty ex:hasMother.
ex:Child owl:equivalentClass _:c.
ex:Felicia rdf:type ex:Child.
```

Using F3, infer:

```
ex:Felicia rdf:type _:c.
```

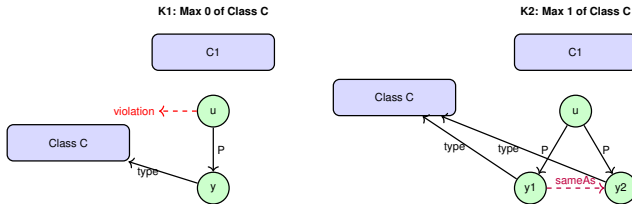
From the triples:

```
_:c owl:maxCardinality "1"^^xsd:nonNegativeInteger;
    owl:onProperty ex:hasMother.
ex:Felicia rdf:type _:c.
ex:Felicia ex:hasMother ex:humanFiona .
ex:Felicia ex:hasMother ex:ogreFiona .
```

Using I1, infer:

```
ex:humanFiona owl:sameAs ex:ogreFiona.
```

Qualified Cardinality Restrictions (Group K)



Qualified cardinality applies only to values of property P that belong to class C .
 Max 0: no such values allowed. | Max 1: if two exist, OWL infers they denote the same individual.

More Axioms

Group K (Qualified Cardinalities)

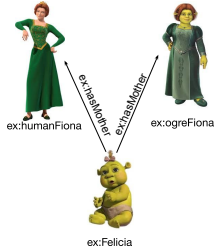
K1. Max Qualified Cardinality Zero:

IF (C1, owl:maxQualifiedCardinality, "0"^^xsd:nonNegativeInteger)
AND (C1, owl:onClass, C) **AND** (u , rdf:type, C1)
AND (u , P, y) **AND** (y , rdf:type, C)
THEN False

K2. Max Qualified Cardinality One:

IF (C1, owl:maxQualifiedCardinality, "1"^^xsd:nonNegativeInteger)
AND (C1, owl:onClass, C) **AND** (u , rdf:type, C1)
AND (u , P, $y1$) **AND** (u , P, $y2$)
THEN ($y1$, owl:sameAs, $y2$)

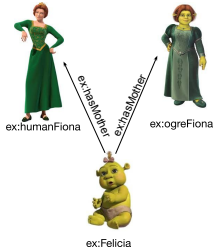
Example owl:maxQualifiedCardinality (1/2)



```

ex:OgreChild owl:equivalentClass
  [owl:maxQualifiedCardinality
    "1"^^xsd:nonNegativeInteger;
    owl:onClass ex:FictionalCharacter;
    owl:onProperty ex:hasMother].
ex:Felicia ex:hasMother ex:humanFiona,
              ex:ogreFiona,
              ex:Fiona.
ex:humanFiona rdf:type ex:FictionalCharacter.
ex:ogreFiona  rdf:type ex:FictionalCharacter.
  
```

Example owl:maxQualifiedCardinality (1/2)



```

ex:OgreChild owl:equivalentClass
  [owl:maxQualifiedCardinality
    "1"^^xsd:nonNegativeInteger;
    owl:onClass ex:FictionalCharacter;
    owl:onProperty ex:hasMother].
ex:Felicia ex:hasMother ex:humanFiona,
            ex:ogreFiona,
            ex:Fiona.
ex:humanFiona rdf:type ex:FictionalCharacter.
ex:ogreFiona  rdf:type ex:FictionalCharacter.
  
```

- owl:maxQualifiedCardinality imposes a restriction on the maximum number of values for a property belonging to a specific class.
- Property: ex:hasMother
- Maximum Qualified Cardinality: 1
- Qualified Class: ex:FictionalCharacter
- The resulting class contains all individuals who have **at most one mother** that are of type ex:FictionalCharacter.
- If an individual has more than the allowed number of qualified values, equivalence must be inferred for the additional values.

Example owl:maxQualifiedCardinality (2/2)

F3: Equivalent Class Instances:

```
IF (C1, owl:equivalentClass, C2)
AND (x, rdf:type, C1)
THEN (x, rdf:type, C2)
```

From the triples:

```
_:c owl:maxQualifiedCardinality
    "1"^^xsd:nonNegativeInteger;
    owl:onClass ex:FictionalCharacter;
    owl:onProperty ex:hasMother.
ex:OgreChild owl:equivalentClass _:c.
ex:Felicia rdf:type ex:OgreChild.
```

Using F3, infer:

```
ex:Felicia rdf:type _:c.
```

J3: Max Qualified Cardinality:

```
IF (C1, owl:maxQualifiedCardinality,
    "1"^^xsd:nonNegativeInteger)
AND
(C1, owl:onClass, D) AND
(C1, owl:onProperty, P) AND
(u, rdf:type, C1) AND
(u, P, y1) AND (y1, rdf:type, D)
AND
(u, P, y2) AND (y2, rdf:type, D)
THEN (y1, owl:sameAs, y2)
```

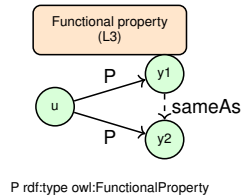
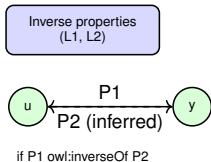
From the triples:

```
_:c owl:maxQualifiedCardinality
    "1"^^xsd:nonNegativeInteger;
    owl:onClass ex:FictionalCharacter;
    owl:onProperty ex:hasMother.
ex:Felicia rdf:type _:c.
ex:Felicia ex:hasMother ex:humanFiona .
ex:Felicia ex:hasMother ex:ogreFiona .
ex:humanFiona rdf:type ex:FictionalCharacter.
ex:ogreFiona rdf:type ex:FictionalCharacter.
```

Using J3, infer:

```
ex:humanFiona owl:sameAs ex:ogreFiona.
```

Intuition: Property Axioms (Group L)



Inverse properties flip the direction of a triple; functional properties force all P -targets of u to be the same individual.

Axioms Properties (1/3)

Group L (Properties)

L1. Inverse Property: IF (P1, owl:inverseOf, P2)
AND (u, P1, y)
THEN (y, P2, u)

L2. Inverse Property: IF (P1, owl:inverseOf, P2)
AND (u, P2, y)
THEN (y, P1, u)

L3. Functional Property: IF (P, rdf:type, owl:FunctionalProperty)
AND (u, P, y1) AND (u, P, y2)
THEN (y1, owl:sameAs, y2)

Examples of Axioms: Group L (Properties)

L1. Inverse Property Example:

- **Axiom:** If $(P1, \text{owl:inverseOf}, P2)$ AND $(u, P1, y)$, THEN $(y, P2, u)$.
- **Example:**
ex:isParentOf owl:inverseOf ex:isChildOf.
ex:Shrek ex:isParentOf ex:Felicia.
- **Inferred:** ex:Felicia ex:isChildOf ex:Shrek.

Examples of Axioms: Group L (Properties)

L1. Inverse Property Example:

- **Axiom:** If $(P1, \text{owl:inverseOf}, P2)$ AND $(u, P1, y)$, THEN $(y, P2, u)$.
- **Example:**
 $\text{ex:isParentOf owl:inverseOf ex:isChildOf}$.
 $\text{ex:Shrek ex:isParentOf ex:Felicia}$.
- **Inferred:** $\text{ex:Felicia ex:isChildOf ex:Shrek}$.

L2. Inverse Property Example:

- **Axiom:** If $(P1, \text{owl:inverseOf}, P2)$ AND $(u, P2, y)$, THEN $(y, P1, u)$.
- **Example:**
 $\text{ex:isMarriedTo owl:inverseOf ex:isSpouseOf}$.
 $\text{ex:Fiona ex:isSpouseOf ex:Shrek}$.
- **Inferred:** $\text{ex:Shrek ex:isMarriedTo ex:Fiona}$.

Examples of Axioms: Group L (Properties)

L1. Inverse Property Example:

- **Axiom:** If $(P1, \text{owl:inverseOf}, P2)$ AND $(u, P1, y)$, THEN $(y, P2, u)$.
- **Example:**
ex:isParentOf owl:inverseOf ex:isChildOf.
ex:Shrek ex:isParentOf ex:Felicia.
- **Inferred:** ex:Felicia ex:isChildOf ex:Shrek.

L2. Inverse Property Example:

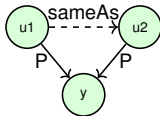
- **Axiom:** If $(P1, \text{owl:inverseOf}, P2)$ AND $(u, P2, y)$, THEN $(y, P1, u)$.
- **Example:**
ex:isMarriedTo owl:inverseOf ex:isSpouseOf.
ex:Fiona ex:isSpouseOf ex:Shrek.
- **Inferred:** ex:Shrek ex:isMarriedTo ex:Fiona.

L3. Functional Property Example:

- **Axiom:** If $(P, \text{rdf:type}, \text{owl:FunctionalProperty})$ AND $(u, P, y1)$ AND $(u, P, y2)$, THEN $(y1, \text{owl:sameAs}, y2)$.
- **Example:**
ex:hasMother rdf:type owl:FunctionalProperty.
ex:Felicia ex:hasMother ex:humanFiona.
ex:Felicia ex:hasMother ex:ogreFiona.
- **Inferred:** ex:humanFiona owl:sameAs ex:ogreFiona.

Intuition: Property Axioms (Group L – 2/3)

Inverse Functional
Property (L4)



P is owl:InverseFunctionalProperty

Irreflexive
Property (L5)



P is owl:IrreflexiveProperty

u P u is forbidden

L4: If two individuals share the same P-value, they must be the same. | L5: No individual can point to itself via P.

Axioms Properties (2/3)

Group L (Properties)

L4. Inverse Functional Property:

IF $(P, \text{rdf:type}, \text{owl:InverseFunctionalProperty})$

AND $(u1, P, y)$ **AND** $(u2, P, y)$

THEN $(u1, \text{owl:sameAs}, u2)$

L5. Irreflexive Property:

IF $(P, \text{rdf:type}, \text{owl:IrreflexiveProperty})$

AND (u, P, u)

THEN False

Examples of Axioms: Group L (Properties)

L4. Inverse Functional Property Example:

- **Axiom:** If $(P, \text{rdf:type, owl:InverseFunctionalProperty})$ AND $(u1, P, y)$ AND $(u2, P, y)$, THEN $(u1, \text{owl:sameAs}, u2)$.
- **Example:**
ex:hasID rdf:type owl:InverseFunctionalProperty.
ex:Fiona ex:hasID ex:ID12345.
ex:Princess ex:hasID ex:ID12345.
- **Inferred:** ex:Fiona owl:sameAs ex:Princess.

Examples of Axioms: Group L (Properties)

L4. Inverse Functional Property Example:

- **Axiom:** If $(P, \text{rdf:type, owl:InverseFunctionalProperty})$ AND $(u1, P, y)$ AND $(u2, P, y)$, THEN $(u1, \text{owl:sameAs}, u2)$.
- **Example:**
ex:hasID rdf:type owl:InverseFunctionalProperty.
ex:Fiona ex:hasID ex:ID12345.
ex:Princess ex:hasID ex:ID12345.
- **Inferred:** ex:Fiona owl:sameAs ex:Princess.

L5. Irreflexive Property Example:

- **Axiom:** If $(P, \text{rdf:type, owl:IrreflexiveProperty})$ AND (u, P, u) , THEN False.
- **Example:**
ex:isParentOf rdf:type owl:IrreflexiveProperty.
ex:Fiona ex:isParentOf ex:Fiona.
- **Result:** Logical inconsistency because a parent cannot be their own parent.

Examples of Axioms: Group L (Properties)

L4. Inverse Functional Property Example:

- **Axiom:** If $(P, \text{rdf:type, owl:InverseFunctionalProperty})$ AND $(u1, P, y)$ AND $(u2, P, y)$, THEN $(u1, \text{owl:sameAs}, u2)$.
- **Example:**
ex:hasID rdf:type owl:InverseFunctionalProperty.
ex:Fiona ex:hasID ex:ID12345.
ex:Princess ex:hasID ex:ID12345.
- **Inferred:** ex:Fiona owl:sameAs ex:Princess.

L5. Irreflexive Property Example:

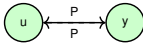
- **Axiom:** If $(P, \text{rdf:type, owl:IrreflexiveProperty})$ AND (u, P, u) , THEN False.
- **Example:**
ex:isParentOf rdf:type owl:IrreflexiveProperty.
ex:Fiona ex:isParentOf ex:Fiona.
- **Result:** Logical inconsistency because a parent cannot be their own parent.

Conclusion:

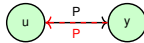
- Inverse Functional Properties ensure uniqueness for object relationships and can infer equivalence of subjects.
- Irreflexive Properties prevent relationships that are logically inconsistent, such as self-references.

Intuition: Property Axioms (Group L – 3/3)

L6: SymmetricProperty

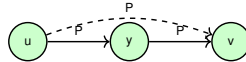


L7: AsymmetricProperty



violates asymmetry

L8: TransitiveProperty



L6: symmetry adds the reverse edge. | L7: asymmetry forbids reverse edges. | L8: transitivity

Axioms Properties (3/3)

Group L (Properties)

L6. Symmetric Property: IF (P, rdf:type, owl:SymmetricProperty)
AND (u, P, y)
THEN (y, P, u)

L7. Asymmetric Property: IF (P, rdf:type, owl:AsymmetricProperty)
AND (u, P, y) AND (y, P, u)
THEN False

L8. Transitive Property: IF (P, rdf:type, owl:TransitiveProperty)
AND (u, P, y) AND (y, P, v)
THEN (u, P, v)

Examples of Axioms: Group L (Properties) (1/2)

L6. Symmetric Property Example:

- **Axiom:** If $(P, \text{rdf:type, owl:SymmetricProperty})$ AND (u, P, y) , THEN (y, P, u) .
- **Example:**
 $\text{ex:isSiblingOf rdf:type owl:SymmetricProperty.}$
 $\text{ex:Fiona ex:isSiblingOf ex:Shrek.}$
- **Inferred:** $\text{ex:Shrek ex:isSiblingOf ex:Fiona.}$

Examples of Axioms: Group L (Properties) (1/2)

L6. Symmetric Property Example:

- **Axiom:** If $(P, \text{rdf:type, owl:SymmetricProperty})$ AND (u, P, y) , THEN (y, P, u) .
- **Example:**
`ex:isSiblingOf rdf:type owl:SymmetricProperty.`
`ex:Fiona ex:isSiblingOf ex:Shrek.`
- **Inferred:** `ex:Shrek ex:isSiblingOf ex:Fiona.`

L7. Asymmetric Property Example:

- **Axiom:** If $(P, \text{rdf:type, owl:AsymmetricProperty})$ AND (u, P, y) AND (y, P, u) , THEN False.
- **Example:**
`ex:isAncestorOf rdf:type owl:AsymmetricProperty.`
`ex:Fiona ex:isAncestorOf ex:Shrek.`
`ex:Shrek ex:isAncestorOf ex:Fiona.`
- **Result:** **Logical inconsistency** because ancestry cannot be reciprocal.

Examples of Axioms: Group L (Properties) (2/2)

L8. Transitive Property Example:

- **Axiom:** If $(P, \text{rdf:type, owl:TransitiveProperty})$ AND (u, P, y) AND (y, P, v) , THEN (u, P, v) .
- **Example:**
`ex:isAncestorOf rdf:type owl:TransitiveProperty.`
`ex:Shrek ex:isAncestorOf ex:Donkey.`
`ex:Donkey ex:isAncestorOf ex:Dragon.`
- **Inferred:** `ex:Shrek ex:isAncestorOf ex:Dragon.`

Examples of Axioms: Group L (Properties) (2/2)

L8. Transitive Property Example:

- **Axiom:** If (P, rdf:type, owl:TransitiveProperty) AND (u , P, y) AND (y , P, v), THEN (u , P, v).
- **Example:**
ex:isAncestorOf rdf:type owl:TransitiveProperty.
ex:Shrek ex:isAncestorOf ex:Donkey.
ex:Donkey ex:isAncestorOf ex:Dragon.
- **Inferred:** ex:Shrek ex:isAncestorOf ex:Dragon.

Conclusion:

- Symmetric Properties ensure bidirectional relationships (e.g., sibling relationships).
- Asymmetric Properties prevent reciprocal relationships, maintaining logical consistency (e.g., ancestry).
- Transitive Properties propagate relationships across chains, ensuring logical completeness (e.g., hierarchical relationships).

More Examples (1/2)



ex:humanFiona	a	ex:HumanFictionalCharacter;
	ex:chosenWifeBy	ex:LordFarquaad;
	ex:isFemaleDescendant	ex:QueenLilian;
	ex:isMomOf	ex:Felicia,ex:Fergues,ex:Farkle.
ex:ogreFiona	a	ex:OgreFictionalCharacter;
	ex:loves	ex:Shrek;
	ex:marries	ex:Shrek;
	ex:isOgreMomOf	ex:Felicia,ex:Fergues,ex:Farkle.
ex:KingHarold	ex:isFatherOf	ex:humanFiona.
ex:QueenLilian	ex:isMotherOf	ex:humanFiona;
	ex:isFemaleDescendant	ex:LiliansMother.
ex:LiliansMother	ex:isMomOf	ex:QueenLilian.
ex:isFemaleDescendant	a	owl:TransitiveProperty,owl:IrreflexiveProperty.
ex:isOgreMomOf	rdfs:subPropertyOf	ex:isMomOf.
ex:isMomOf	a	owl:InverseFunctionalProperty.

More Examples (2/2)

C3: SubProperty Inference

IF (P1, `rdfs:subPropertyOf`, P2) AND
 (a, P1, b)
THEN (a, P2, b)

L8: Transitive Property

IF (P, `rdf:type`, `owl:TransitiveProperty`;
AND (u, P, y) AND (y, P, v)
THEN (u, P, v)

E3: `owl:sameAs` Property

IF (x, `owl:sameAs`, y) AND (x, P, z)
THEN (y, P, z)

Relevant Triples and Inferences

Relevant Triples:

```
ex:ogreFiona ex:isMomOf ex:Felicia.  
ex:humanFiona ex:isMomOf ex:Felicia.  
ex:humanFiona owl:sameAs ex:ogreFiona.  
ex:humanFiona ex:isFemaleDescendant ex:LiliansMother.
```

Using C3 (three times), infer:

```
ex:ogreFiona ex:isMomOf ex:Felicia.
```

Using L8, infer:

```
ex:humanFiona ex:isFemaleDescendant ex:LiliansMother.
```

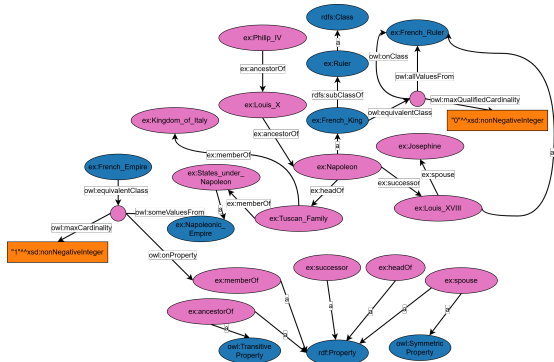
Using E3, infer:

```
ex:ogreFiona ex:isFemaleDescendant ex:LiliansMother.
```

Example: OWL Axioms



- Demonstrates OWL properties
- Will use this to explore OWL Axioms



Example OWL graph: French Emperor
Napoleon Bonaparte

Exercise (1/2)

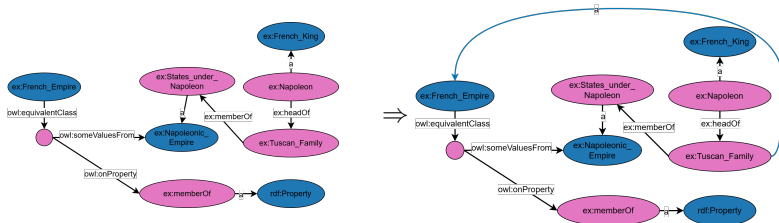
Participate in the survey at **Stud.IP**

Group	Identifier	Axiom Description
SameAs	E1	Symmetry: If $x \text{ owl:sameAs } y$, then $y \text{ owl:sameAs } x$.
	E2	Transitivity: If $x \text{ owl:sameAs } y$ and $y \text{ owl:sameAs } z$, then $x \text{ owl:sameAs } z$.
	E3	Equivalent Subject: If $x \text{ owl:sameAs } y$ and (x, P, z) , then (y, P, z) .
	E4	Equivalent Object: If $x \text{ owl:sameAs } y$ and (z, P, x) , then (z, P, y) .
Equivalent Classes	F1	Symmetry: If $C1 \text{ owl:equivalentClass } C2$, then $C2 \text{ owl:equivalentClass } C1$.
	F2	Transitivity: If $C1 \text{ owl:equivalentClass } C2$ and $C2 \text{ owl:equivalentClass } C3$, then $C1 \text{ owl:equivalentClass } C3$.
	F3	Equivalent Class Instances: If $C1 \text{ owl:equivalentClass } C2$ and $(x, \text{rdf:type}, C1)$, then $(x, \text{rdf:type}, C2)$.
Quantified Values	H1	Universal Quantification: If $C1 \text{ owl:allValuesFrom } C2$, then all values of property P for $C1$ must be in $C2$.
	H2	Existential Quantification: If $C1 \text{ owl:someValuesFrom } C2$, then $C1$ has at least one value of property P in $C2$.
	H3	Instantiation: If $C1 \text{ owl:someValuesFrom } C2$ and (u, P, v) with $v \in C2$, then $u \in C1$.

Exercise (2/2)

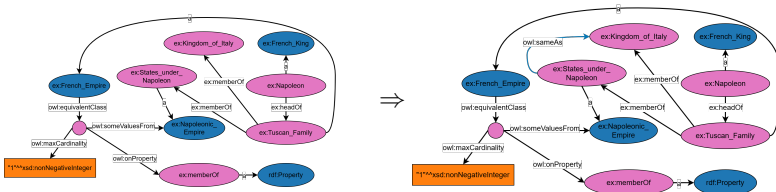
Group	Identifier	Axiom Description
Cardinalities	J1	Max Cardinality One: If $C1$ owl:maxCardinality 1, then an individual in $C1$ has at most one value for property P .
	K2	Qualified Cardinality: If $C1$ owl:maxQualifiedCardinality 1 on C , then $C1$ has at most one value for property P in class C .
Properties	L1	Inverse Property: If $P1$ owl:inverseOf $P2$ and $(u, P1, y)$, then $(y, P2, u)$.
	L3	Functional Property: If P rdf:type owl:FunctionalProperty, then for any $u, P, y1$ and $u, P, y2$, $y1 = y2$.
	L8	Transitive Property: If P rdf:type owl:TransitiveProperty, then for (u, P, y) and (y, P, v) , infer (u, P, v) .

Example: OWL Axioms Group H



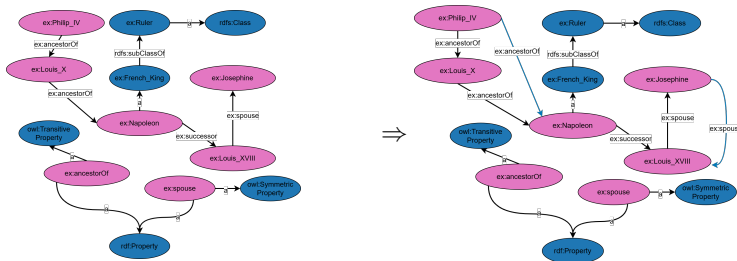
- Axiom- H3 (Instantiation Existential Quantification) Example:**
 IF `ex:French_Empire owl:someValuesFrom ex:Napoleonic_Empire`
 AND `ex:French_Empire owl:onProperty ex:memberOf`
 AND `ex:Tuscan_Family ex:memberOf ex:States_under_Napoleon`
 AND `ex:States_under_Napoleon a ex:Napoleonic_Empire`
 THEN `ex:Tuscan_Family a ex:French_Empire`.

Example: OWL Axioms Group J



- Axiom- J2 (Max Cardinality One) Example:**
 IF `ex:French_Empire owl:maxCardinality "1" ^ xsd:nonNegativeInteger`
 AND `ex:French_Empire owl:onProperty ex:memberOf`
 AND `ex:Tuscan_Family a ex:French_Empire`
 AND `ex:Tuscan_Family ex:memberOf ex:States_under_Napoleonic`
 AND `ex:Tuscan_Family ex:memberOf ex:Kingdom_of_Italy`
 THEN `ex:States_under_Napoleonic owl:sameAs ex:Kingdom_of_Italy`.

Example: OWL Axioms Group L



- Axiom- L6 (Symmetric Property) Example:**
 IF `ex:spouse` `rdf:type` `owl:SymmetricProperty`
 AND `ex:Louis_XVIII` `ex:spouse` `ex:Josephine`
 THEN `ex:Josephine` `ex:spouse` `ex:Louis_XVIII`.
- Axiom- L8 (Transitive Property) Example:**
 IF `ex:ancestorOf` `rdf:type` `owl:TransitiveProperty`
 AND `ex:Philip_IV` `ex:ancestorOf` `ex:Louis_X`
 AND `ex:Louis_X` `ex:ancestorOf` `ex:Napoleon`
 THEN `ex:Philip_IV` `ex:ancestorOf` `ex:Napoleon`.

Example from Slide 15

#Classes

```
ex:ParentLivingBeing a rdfs:Class.
ex:LivingBeing a rdfs:Class.
ex:Ogre a rdfs:Class.
ex:FictionalCharacter a rdfs:Class.
ex:Human a rdfs:Class.
```

#SubClasses

```
ex:Ogre rdfs:subClassOf ex:LivingBeing.
ex:FictionalCharacter rdfs:subClassOf
    ex:LivingBeing.
ex:Human rdfs:subClassOf
    ex:LivingBeing.
ex:ParentLivingBeing rdfs:subClassOf
    ex:LivingBeing.
```

#Properties

```
ex:hasChild a rdf:Property;
    rdfs:domain ex:ParentLivingBeing;
    rdfs:range ex:LivingBeing.
```

#Instances of Classes

```
ex:c0 a ex:ParentLivingBeing.
ex:c1 a ex:ParentLivingBeing.
ex:c2 a ex:ParentLivingBeing.
ex:c3 a ex:ParentLivingBeing.
```

```
ex:d0 a ex:Ogre.
ex:d1 a ex:Ogre.
ex:d2 a ex:Ogre.
ex:d2 a ex:FictionalCharacter.
ex:d3 a ex:FictionalCharacter.
ex:d4 a ex:FictionalCharacter.
ex:d5 a ex:Human.
ex:d6 a ex:Human.
```

#Instances of Properties

```
ex:c0 ex:hasChild ex:d0, ex:d1.
ex:c1 ex:hasChild ex:d0, ex:d1, ex:d2,
    ex:d3, ex:d4, ex:d5.
ex:c2 ex:hasChild ex:d6.
ex:c3 ex:hasChild ex:d0, ex:d1, ex:d2.
```

Universal and Existential Quantifiers

Axioms for Trading of Universal Quantification

$$\forall x (P) \equiv \forall x (P1 \Rightarrow P2) \equiv \forall x (\neg P1 \vee P2)$$

Generalized De Morgan

$$\forall x (P1 \Rightarrow P2) \equiv \neg \exists x (P1 \wedge \neg P2)$$

SPARQL Query Examples (1/4)

Q1) Who are the parents (i.e., those that participate in `ex:hasChild`) who have at least one ogre child?

```
PREFIX ex: <http://www.example.org/>
SELECT DISTINCT ?parent
WHERE {
    ?parent ex:hasChild ?child.
    ?child a ex:Ogre.
}
```

Answers

```
{ (?parent, ex:c0),
  (?parent, ex:c1),
  (?parent, ex:c3)}
```

SPARQL Query Examples (2/4)

Q2) Who are the parents (i.e., those that participate in `ex:hasChild`) whose children are all ogres?

```
PREFIX ex: <http://www.example.org/>
SELECT DISTINCT ?parent
WHERE {
    ?parent ex:hasChild ?child.
    ?child a ex:Ogre.
    FILTER NOT EXISTS {
        ?parent ex:hasChild ?child1.
        ?child1 a ?type1.
        FILTER (?child1 != ?child)
        FILTER (?type1 NOT IN (ex:Ogre, ex:LivingBeing))
    }
}
```

Answers

```
{(?parent, ex:c0),
 (?parent, ex:c3)}
```

Explanation of Query Q2 (1/2)

Query Q2: Find parents whose children are all ogres.

1. Universal Quantifier Translation:

$$\forall x(\text{parent hasChild } x \Rightarrow x \text{ is Ogre})$$

This states that for every child of a parent, the child must be an Ogre.

2. Generalized De Morgan's Law:

$$\forall x(P1 \Rightarrow P2) \equiv \neg \exists x(P1 \wedge \neg P2)$$

In this case:

$$\neg \exists x(\text{parent hasChild } x \wedge x \text{ is not Ogre}).$$

This means no child of the parent exists that is not an Ogre.

Explanation of Query Q2 (2/2)

Query Q2: Find parents whose children are all ogres.

SPARQL Query Explanation

- `?parent ex:hasChild ?child. ?child a ex:Ogre.`
 - Retrieves parents who have at least one child that is an Ogre ($\exists$).
- `FILTER NOT EXISTS {...}`
 - Ensures there are no children of the parent that are not Ogres ($\neg\exists$).

Conclusion

The combination of $\exists$ and $\forall$ through De Morgan's law ensures that only parents whose children are all Ogres are included in the query result.

Answer: `{(?parent, ex:c0), (?parent, ex:c3)}`

SPARQL Query Examples (3/4)

Q3) Who are the parents (i.e., those that participate in `ex:hasChild`) whose children are all humans?

```
PREFIX ex: <http://www.example.org/>
SELECT DISTINCT ?parent
WHERE {
  ?parent ex:hasChild ?child.
  ?child a ex:Human.
  FILTER NOT EXISTS {
    ?parent ex:hasChild ?child1.
    ?child1 a ?type1.
    FILTER (?child1 != ?child)
    FILTER (?type1 NOT IN (ex:Human, ex:LivingBeing))
  }
}
```

Answers

```
{(?parent, ex:c2)}
```

Explanation of Query Q3 (1/2)

Query Q3: Find parents whose children are all humans.

1. Universal Quantifier Translation:

$$\forall x(\text{parent hasChild } x \Rightarrow x \text{ is Human})$$

This states that for every child of a parent, the child must be Human.

2. Generalized De Morgan's Law:

$$\forall x(P1 \Rightarrow P2) \equiv \neg \exists x(P1 \wedge \neg P2)$$

In this case:

$$\neg \exists x(\text{parent hasChild } x \wedge x \text{ is not Human}).$$

This means no child of the parent exists that is not Human.

Explanation of Query Q3 (2/2)

Query Q3: Find parents whose children are all humans.

SPARQL Query Explanation

- `?parent ex:hasChild ?child. ?child a ex:Human.`
 - Retrieves parents who have at least one child that is Human ($\exists$).
- `FILTER NOT EXISTS {...}`
 - Ensures there are no children of the parent that are not Humans ($\neg\exists$).

Conclusion

The combination of $\exists$ and $\forall$ through De Morgan's law ensures that only parents whose children are all Humans are included in the query result.

Answer: `{(?parent, ex:c2)}`

SPARQL Query Examples (4/4)

Q4) Who are the parents living beings (i.e., those which are instances of the class `ex:ParentLivingBeing`) whose children are all ogres?

PREFIX `ex: <http://www.example.org/>`

```
SELECT DISTINCT ?parent
WHERE {
  { SELECT DISTINCT ?parent
    WHERE {
      ?parent ex:hasChild ?child.
      ?child a ex:Ogre.
      FILTER NOT EXISTS {
        ?parent ex:hasChild ?child1.
        ?child1 a ?type1.
        FILTER (?child1 != ?child)
        FILTER (?type1 NOT IN (ex:Ogre, ex:LivingBeing))}}
    UNION
    { SELECT DISTINCT ?parent
      WHERE {
        ?parent a ex:ParentLivingBeing.
        FILTER NOT EXISTS {
          ?parent ex:hasChild ?child.}}}}
```

Answers

```
{ (?parent, ex:c0),
  (?parent, ex:c3),
  (?parent, ex:c4)}
```

Explanation of Query Q4 (1/2)

Query Q4: Find parents living beings whose children are all ogres.

Objective: Retrieve parents whose children are all ogres.

- `?parent ex:hasChild ?child. ?child a ex:Ogre.`
 - Retrieves parents who have at least one child that is an ogre ($\exists$).
- `FILTER NOT EXISTS {...}`
 - Ensures no child exists such that:
 - The child is not an ogre, or
 - The child is not a living being.
 - Logic Translation:

$$\forall x(\text{parent hasChild } x \Rightarrow x \text{ is Ogre or LivingBeing}).$$

- Generalized De Morgan:

$$\neg \exists x(\text{parent hasChild } x \wedge x \text{ is not Ogre or LivingBeing}).$$

Explanation of Query Q4 (2/2)

Query Q4: Find parents living beings whose children are all ogres.

Objective: Include parents that belong to the class `ex:ParentLivingBeing` but do not participate in the property `ex:hasChild`.

- `?parent a ex:ParentLivingBeing.`
 - Retrieves all individuals belonging to the class `ex:ParentLivingBeing`.
- `FILTER NOT EXISTS {...}`
 - Ensures no relationship exists between the parent and any child using `ex:hasChild`.
 - Logic Translation:
$$\neg \exists x(\text{parent hasChild } x).$$
 - This ensures the inclusion of parents who are living beings but do not have any children.

Final Union Result

Union:

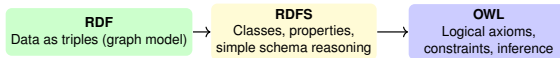
- Combines results from the first subquery (parents whose children are all ogres) and the second subquery (parents who are living beings but have no children).

Final Answer: $\{(?parent, ex:c0), (?parent, ex:c3)\}$

Comparison of RDFS and OWL

RDFS	OWL
Schema language for describing classes, properties, domain and range	Ontology language with rich logical constructors (e.g., equivalence, intersections, restrictions)
Supports the Open World Assumption	Supports the Open World Assumption with more expressive constraints (e.g., cardinalities, negation)
RDFS axioms allow simple inferences (subClassOf, subPropertyOf, domain, range)	OWL axioms support advanced reasoning: deducing new facts, detecting inconsistencies, identifying False statements

Lecture Summary – Key Takeaways



Inference: derive missing facts

Open World Assumption: unknown $\neq$ false

Consistency: detect contradictions

Overall Message

RDF provides a graph-based data model. RDFS adds lightweight schema knowledge. OWL adds expressive axioms that enable automated reasoning to infer new knowledge or detect inconsistencies.